

RELATORIO DE AUDITORIA DE SEGURANCA

Relatorio de Auditoria de Seguranca
Validador eSocial

Versao corrigida: v1.1-alpha | Data do relatorio: 23/09/2026

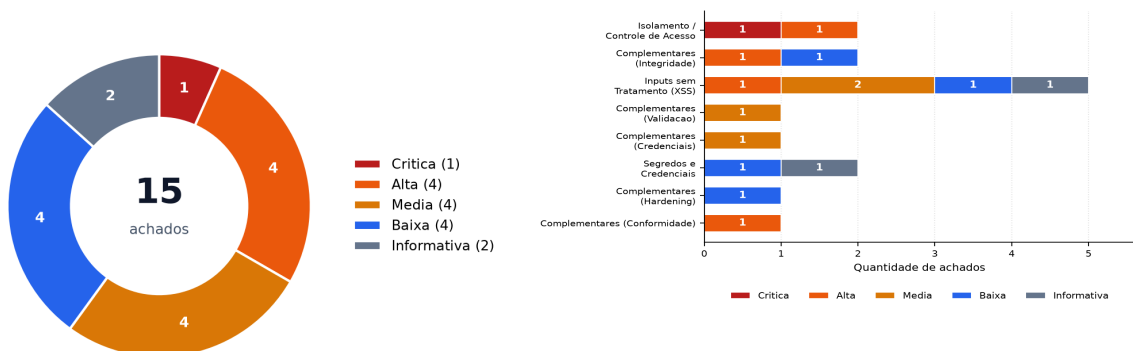
Repositorio	github.com/forg3/validador-esocial (modulo esocial-emissor-livre) - release v1.1-alpha
Escopo auditado	Codigo Go (cmd/, internal/), 21 templates HTML, JavaScript embutido, CSS, Dockerfile, workflow GitHub Actions, go.mod/go.sum, .gitignore, README e demais documentos, alem do historico git completo (git log --all -p).
Volume analisado	5.572 linhas de Go, 21 templates HTML, 1 workflow de CI, 1 Dockerfile, 2 arquivos de configuracao.
Metodo	Revisao manual linha a linha de todos os handlers de rota, camada de persistencia, geradores de XML, camada criptografica e templates; varredura automatizada por padroes de segredo e sinks de XSS; sondas executadas com httpptest contra as rotas reais e com o runtime Go 1.26.8 instalado.
Total de achados	15 (1 critica, 4 altas, 4 medias, 5 baixas, 1 informativa) - TODOS CORRIGIDOS

Nota metodologica: as cinco categorias solicitadas foram mapeadas para a stack detectada - **Go 1.26.8 (net/http + html/template), HTMX 2.x, SQLite via modernc.org/sqlite, criptografia PKCS#12/XMLDSig propria, sem ORM, sem framework de auth, deploy via Dockerfile + GitHub Actions (GHCR)**. Assim: (1) isolamento de inquilino/dono equivale a middleware de tenant/ownership - inexistente, avaliado como ausencia de fronteira de autorizacao; (2) permissao definida no navegador equivale a gates de papel - inexistentes, pois nao ha papeis; (3) IDOR foi auditado em todos os handlers que recebem ID em path; (4) segredos expostos cobriram codigo, configs, Dockerfile, CI, docs e historico git; (5) XSS cobriu templates, JavaScript embutido e, no backend, a saida HTML manual (fmt.Fprintf) que substitui o template.

1. Resumo executivo

A auditoria cobriu todo o código do **Validador eSocial v1.1-alpha**: 15 achados verificados, sendo **1 crítico**, **4 altos**, **4 médios**, **5 baixos** e **1 informativo**. O risco central é a **ausência total de autenticação e autorização** combinada ao bind em **0.0.0.0:8000**, que transforma um aplicativo pensado para uso local em um painel aberto para qualquer host da rede - com leitura de CPFs, troca de certificado digital e exclusão de dados. Em segundo plano, o produto **simula** assinatura digital e transmissão ao eSocial e grava recibos oficiais fictícios, criando uma trilha de auditoria falsa. A base técnica, porém, é sólida nos pontos estruturais: 100% das consultas SQL são parametrizadas, os templates usam escaping contextual do html/template, não há segredos hardcoded (nem no histórico git) e não há path traversal, command injection nem XXE exploráveis. **Todos os achados foram corrigidos nesta revisão** (ver seção 7, com a evidência de cada correção); a validação por schema, agora ligada, revelou ainda um achado adicional de conformidade (M-05) que permanece aberto por depender da adequação dos leiautes.

Severidade	Qtd.	Achados	Leitura de risco
CRITICA	1	C-01	Corrigir antes de qualquer uso em rede/Docker.
ALTA	4	A-01, A-02, A-03, M-05	Corrigir no próximo ciclo curto; explorável hoje.
MEDIA	4	M-01, M-02, M-03, M-04	Corrigir antes de uso em produção real.
BAIXA	4	B-01, B-02, B-03, B-04	Hardening planejado.
INFORMATIVA	2	B-05, I-01	Acompanhar em manutenção.
TOTAL	15		



Figuras 1 e 2 - Distribuição por severidade (rosca) e por categoria (barras empilhadas).

2. Pontos fortes e pontos fracos

2.1 Pontos fortes (verificados)

✓ Escaping contextual do html/template em todos os templates (sem uso de template.HTML)

internal/web/templates/*.html, internal/web/server.go:97-99 - Sonda executada com os padroes reais do projeto (onclick="f('{{ .Nome }})", href="/fila/{{ .ID }}/xml", <textarea>{{ .XML }}</textarea>"): payloads de quebra de string JS, de injecao de HTML e de path traversal foram neutralizados (\u0027, \u003c, %22%3e) e o XML apareceu como <eSocial>. Nenhum template usa template.HTML (a funcao safeHTML esta morta).

✓ Nenhuma consulta SQL montada por concatenacao - 100% parametrizada

internal/storage/storage.go:149-152, 236-239, 253-265, 279-290, 342-349, 369, 377, 386-394 - Todas as operacoes usam placeholders '?' via database/sql; grep por Sprintf/concatenacao em storage.go nao retorna nada. Nao ha superficie de SQL injection.

✓ Sem segredos hardcoded e historico git limpo

Dockerfile, .github/workflows/package.yml:35, .gitignore:11-22, .opencode/.mcp.json - Varredura por api_key/secret/password/token/BEGIN PRIVATE KEY no codigo, configs, CI, docs e em todo o historico (git log --all -p) nao encontrou segredos. O CI usa \${ secrets.CR_PAT || secrets.GITHUB_TOKEN } e o .gitignore exclui *.pfx, *.p12, *.pem, *.key, *.db.

✓ Upload de certificado imune a path traversal

internal/web/server.go:412-414 + mime/multipart/multipart.go:101 (stdlib Go 1.26.8) - O nome enviado passa por filepath.Base dentro do proprio stdlib (comprovado no fonte do Go instalado), portanto './../x.pfx' nao escapa de ./dados/certificados. O diretorio e criado com 0700.

✓ Sem command injection

internal/esocial/validator.go:144-158 - O unico exec.Command do projeto e 'xmllint --noout --schema <xsd> <tmp>' com caminho XSD fixo (switch interno) e arquivo temporario gerado pelo servidor (os.CreateTemp); nenhum argumento vem do usuario.

✓ XXE e expansao de entidades bloqueados pelo encoding/xml

internal/web/server.go:1261, internal/web/server.go:1051-1067, internal/esocial/validator.go:110 - Sonda executada: payload com <!ENTITY xxe SYSTEM "file:///etc/passwd"> e payload de expansao de entidades customizadas retornaram 'invalid character entity' - o decoder Go nao resolve DTD/entidades externas.

✓ Cliente SOAP com mTLS e TLS minimo 1.2, sem InsecureSkipVerify

internal/soap/client.go:116-136 - tls.Config com Certificates do A1, MinVersion TLS 1.2, http.Client com timeout de 60s e transporte HTTP/1.1. Ressalva: o cliente nao esta conectado a nenhuma rota (ver achado A-02).

✓ Frontend sem sinks perigosos de HTML/JS

internal/web/templates/editor_s2240.html:261, evento_s2240.html:261, menu_eventos.html:90 - Os unicos usos de innerHTML sao limpeza de container (= ") e configuracao de swap do HTMX; nao ha eval, new Function, document.write nem markdown/HTML renderizado de usuario. Nao existe lib de sanitizacao porque nao ha caminho de renderizacao de HTML cru - os pontos de HTML manual sao o achado A-03.

✓ Cabecalho de resposta imune a injecao CRLF

net/http/header.go:139,206 (stdlib Go 1.26.8) + internal/web/server.go:1404 - O nome do arquivo em Content-Disposition e derivado do ID do evento, mas o net/http substitui \r e \n por espaco em valores de cabecalho, impedindo header injection mesmo com ID controlado pelo usuario (achado B-02).

2.2 Pontos fracos (riscos centrais)

- **Fronteira de confianca inexistente:** o servidor nao identifica quem faz a requisicao - nao ha login, sessao, token nem papeis. Toda a aplicacao opera como um unico usuario administrador implicito (C-01).
- **Escrita sem verificacao de intencao:** nenhum endpoint de escrita exige prova de que a requisicao veio da propria interface (CSRF, A-01).

- **Integridade documental comprometida:** assinatura e transmissao simuladas com recibos ficticios apresentados como oficiais (A-02) e campos sem escape no XML gerado (M-01).
- **Saida HTML manual:** o handler de certificado escreve HTML por `fmt.Fprintf`, contornando o escaping automatico do `html/template` (A-03).
- **Ausencia de defesa em profundidade:** sem limites de tamanho (M-02), sem rate limit (M-04), sem cabecalhos de seguranca (B-03) e com erros de persistencia silenciados (B-04).

3. Tabela de achados

A tabela abaixo resume todos os achados verificados. O detalhamento com trecho de código, explorabilidade, impacto e correção sugerida está na seção 4.

Sev.	ID	Arquivo:linha	Descrição
CRITICA	C-01	cmd/server/main.go:55 + internal/web/server.go:198-257	Aplicação sem autenticação alguma, ouvindo em todas as interfaces de rede (0.0.0.0:8000) Isolamento e Controle de Acesso
ALTA	A-01	internal/web/server.go:213-251 (rotas) + internal/web/templates/*.html (formularios)	CSRF: endpoints de escrita sem token anti-CSRF e sem validação de Origin/Referer Isolamento e Controle de Acesso
ALTA	A-02	internal/web/generator.go:484-533, internal/web/server.go:1444-1543, README.md:14,65-67	Assinatura digital e recibos do eSocial são FALSIFICADOS (simulação) e apresentados como oficiais Complementares (Integridade)
ALTA	A-03	internal/web/server.go:472-475, 489-492, 518-539	HTML injection / XSS armazenado em POST /configuração/testar (saída sem escape via fmt.Fprintf) Inputs sem Tratamento (XSS)
MEDIA	M-01	internal/web/generator.go:118, 120, 129, 135, 140, 166, 176, 255, 257, 307, 334, 398, 401, 414	XML injection: campos de formulário interpolados sem escape nos geradores de eventos Inputs sem Tratamento (XSS)
MEDIA	M-02	internal/web/server.go:409-441, 1051-1067, 701-705	Uploads e corpos de requisição sem limite de tamanho (consumo de memória/disco) Inputs sem Tratamento (XSS)
MEDIA	M-03	internal/web/server.go:1433-1436 + internal/web/generator.go:424-482 + internal/esocial/validator.go:101	Validação XSD real existe no projeto mas não está ligada ao handler; UI afirma validação oficial Complementares (Validação)
MEDIA	M-04	internal/web/server.go:214-216, 460-493	Força bruta da senha do certificado A1 sem autenticação e sem rate limit Complementares (Credenciais)
BAIXA	B-01	internal/web/server.go:413-416 + Dockerfile:16	Permissões frouxas: PFX gravado com 0644 e diretório de dados 777 na imagem Docker Segredos e Credenciais
BAIXA	B-02	internal/web/server.go:1277-1284	ID do evento extraído do XML do usuário sem validação de formato Inputs sem Tratamento (XSS)
BAIXA	B-03	internal/web/server.go:198-257 (Rotas) - nenhum middleware de cabeçalhos	Ausência de cabeçalhos de segurança (CSP, X-Content-Type-Options, X-Frame-Options, Referrer-Policy) Complementares (Hardening)
BAIXA	B-04	internal/web/server.go:660, 966, 1045, 1113, 1297, 1502	Erros de escrita/exclusão são descartados e a interface informa sucesso Complementares (Integridade)
INFORMATIVA	B-05	internal/web/server.go:97-99	Função de template safeHTML (template.HTML) definida e não utilizada Inputs sem Tratamento (XSS)
ALTA	M-05	internal/web/generator.go:97-181, 239-342, 381-421 + internal/data/xsd/*.xsd	XML gerado pelo editor não é aderente ao XSD oficial S-1.3 (revelado ao ligar a validação real) Complementares (Conformidade)
INFORMATIVA	I-01	internal/crypto/cert_test.go:66, 125	Senhas de teste embutidas no código (nenhum segredo de produção encontrado) Segredos e Credenciais

4. Achados detalhados por categoria

Isolamento e Controle de Acesso

C-01 - Aplicacao sem autenticao alguma, ouvindo em todas as interfaces de rede (0.0.0.0:8000)

CRITICA

Local: cmd/server/main.go:55 + internal/web/server.go:198-257 | **Categoria:** Isolamento e Controle de Acesso

```
Addr: fmt.Sprintf(":%d", *porta) // main.go:55 -> bind 0.0.0.0
mux := http.NewServeMux()         // server.go:199
// nenhum middleware de sessao/token/login em Rotas()
```

Por que e exploravel: O servidor HTTP nao possui qualquer camada de autenticao ou autorizacao: nao existe login, sessao, cookie, token, middleware de auth ou validacao de origem. O bind e feito em ":8000", que em Go escuta em todas as interfaces (0.0.0.0), e nao em 127.0.0.1. Sonda executada contra as rotas reais retornou HTTP 200 sem credencial alguma para GET /fila, GET /colaboradores, GET /configuracao, DELETE /colaboradores/{id} e POST /fila/lote/transmitir. Qualquer host na mesma rede (Wi-Fi corporativo, LAN, container com porta publicada) le nomes e CPFs de trabalhadores, baixa XMLs, altera a configuracao da empresa, troca o certificado digital, dispara assinatura/transmissao em lote e apaga registros.

Impacto: Exposicao de dados pessoais (LGPD) de toda a folha de colaboradores, manipulacao de documentos fiscais assinados, substituicao do certificado A1 e destruicao de dados - tudo sem credencial.

Condicao de explorabilidade: Exploravel sempre que a porta estiver acessivel (LAN, docker com -p 8000:8000, notebook em rede publica).

Correcao sugerida: 1) Trocar o bind para 127.0.0.1 por padrao (flag explicita para expor). 2) Introduzir autenticao (sessao com cookie SameSite=Strict ou token local) com middleware aplicado a todas as rotas exceto /static. 3) Validar o cabeçalho Host (allowlist localhost/127.0.0.1) para mitigar DNS rebinding. 4) Documentar o modo Docker como exposto apenas a rede confiavel, com auth obrigatoria.

A-01 - CSRF: endpoints de escrita sem token anti-CSRF e sem validacao de Origin/Referer

ALTA

Local: internal/web/server.go:213-251 (rotas) + internal/web/templates/*.html (formularios) | **Categoria:** Isolamento e Controle de Acesso

```
mux.HandleFunc("POST /configuracao", s.handleSalvarConfiguracao)
mux.HandleFunc("POST /fila/lote/transmitir", s.handleTransmitirLote)
mux.HandleFunc("POST /colaboradores", s.handleSalvarColaborador)
// grep -rni "csrf" => nenhum resultado no repositorio
```

Por que e exploravel: Nenhum formulario possui campo anti-CSRF (grep por 'csrf' no repositorio nao retorna nada) e nenhum handler verifica Origin/Referer. Como os POSTs sao requisicoes simples, uma pagina maliciosa visitada pelo operador pode submeter formularios cross-site para localhost:8000 e executar acoes de escrita sem que o usuario perceba: alterar razao social/CNPJ/ambiente, cadastrar colaborador, assinar e transmitir lotes inteiros. Os metodos DELETE nao sao alcancaveis por formulario (exigiriam preflight CORS), mas todos os POSTs sensiveis sao.

Impacto: Integridade de configuracao e de dados fiscais; alteracao de ambiente (Producao Restrita para Producao Oficial).

Condicao de explorabilidade: Requer que o operador visite uma pagina controlada pelo atacante enquanto o servico esta ativo.

Correcao sugerida: Emitir token sincronizado por formulario (campo hidden) validado no servidor para todo metodo nao-idempotente e/ou exigir cabeçalho customizado (X-Requested-With / X-CSRF-Token) com validacao de Origin/Referer. Se houver sessao, usar cookie SameSite=Strict + verificacao de Host.

Complementares (Integridade)

A-02 - Assinatura digital e recibos do eSocial sao FALSIFICADOS (simulacao) e apresentados como oficiais

ALTA

Local: internal/web/generator.go:484-533, internal/web/server.go:1444-1543, README.md:14,65-67 | **Categoria:** Complementares (Integridade)

```
<SignatureValue>SIMULATED_RSA_SHA256_SIGNATURE_%s</SignatureValue> // generator.go:503
recibo = fmt.Sprintf("%d.2.%s.%014d", ambiente, agora, nano+107) // generator.go:521
"Recibo oficial do eSocial ja confirmado: %s" // server.go:1491
// internal/soap/client.go nunca e referenciado fora dos testes (codigo morto)
```

Por que e exploravel: Os handlers de assinatura e transmissao nao usam criptografia nem o webservice: SimularAssinatura gera um envelope XMLDSig falso (DigestValue calculado sobre o proprio texto e SignatureValue literal 'SIMULATED_RSA_SHA256_SIGNATURE_'), e SimularTransmissao inventa protocolo/recibo com a mensagem 'Recibo emitido pelo Serpro/Receita Federal'. O status do evento passa a 'aceito' e a interface afirma 'Recibo oficial do eSocial ja confirmado'. O cliente SOAP mTLS real existe (internal/soap/client.go) mas nao esta conectado a nenhuma rota. O README afirma transmissao real pelos webservices oficiais.

Impacto: Trilha de auditoria fiscal falsa: o usuario acredita ter cumprido obrigacao acessoria do eSocial. Risco documental, regulatorio e de conformidade (declaracao inexistente), alem de assinatura sem valor juridico.

Condicao de explorabilidade: Sempre ativo nesta versao: qualquer evento 'assinado/aceito' hoje e ficticio.

Correcao sugerida: Rotular explicitamente o modo simulacao na interface e no banco (ex.: status 'simulado'), bloquear a geracao de recibo quando nao houver transmissao real e conectar o ClienteSOAP (mTLS + certificado A1) aos handlers, persistindo o XML de resposta oficial. Ajustar o README para refletir o estado real.

B-04 - Erros de escrita/exclusao sao descartados e a interface informa sucesso

BAIXA

Local: internal/web/server.go:660, 966, 1045, 1113, 1297, 1502 | **Categoria:** Complementares (Integridade)

```
id := r.PathValue("id")
_ = s.db.ExcluirEvento(id) // server.go:1502 (erro ignorado)
s.handleFilaComFlash(w, r, fmt.Sprintf("Evento %s removido da fila.", id), false)
```

Por que e exploravel: Diversos handlers ignoram o erro das operacoes de persistencia (SalvarEvento/SalvarColaborador/Excluir*) e sempre exibem mensagem de sucesso. Em caso de falha de disco ou violacao de constraint, o usuario acredita que o dado foi gravado/excluido. E o mesmo padrao em handleSalvarS2210 (966), handleSalvarS2220 (1045), handleImportarXMLASO (1113), handleSalvarEventoGenerico (1297) e handleExcluirColaborador (660).

Impacto: Trilha de auditoria enganosa e perda silenciosa de dados.

Condicao de explorabilidade: Ocorre em qualquer falha de I/O ou de constraint.

Correcao sugerida: Tratar o erro retornado, exibir flash de erro e retornar HTTP 4xx/5xx coerente (404 quando o registro nao existe).

Inputs sem Tratamento (XSS)**A-03 - HTML injection / XSS armazenado em POST /configuracao/testar (saida sem escape via fmt.Fprintf)**

ALTA

Local: internal/web/server.go:472-475, 489-492, 518-539 | **Categoria:** Inputs sem Tratamento (XSS)

```
fmt.Fprintf(w, `<div class="flash flash-err" ...>
  <strong>Arquivo nao encontrado:</strong> O arquivo %s ...
</div>`, filepath.Base(cfg.CertificadoPath)) // server.go:472-474
```

Por que e exploravel: O handler escreve HTML concatenando dados controlados pelo usuario diretamente em fmt.Fprintf, sem usar o html/template (que escaparia). cfg.CertificadoPath vem de header.Filename do upload de certificado e fica persistido no SQLite; o nome do arquivo e tambem o CN/titular do certificado sao refletidos nos

blocos das linhas 518-539. Sonda executada via httptest contra a rota real: configurando CertificadoPath = '.pfx' o corpo da resposta 200 conteve a tag intacta (HTML_INJECTION_CONFIRMED). Como o valor persiste no banco, o script executa em cada visita a pagina de certificado (XSS armazenado).

Impacto: Execucao de JavaScript na origem do painel (localhost:8000): leitura de todo o conteudo do painel, disparo de acoes autenticadas pelo proprio navegador do operador e exfiltracao de dados de colaboradores.

Condicao de explorabilidade: Requer que um arquivo .pfx com nome/atributos maliciosos seja carregado (ou via CSRF do achado A-01).

Correcao sugerida: Substituir todo fmt.Fprintf de HTML por template.ExecuteTemplate com html/template (escaping contextual) ou aplicar template.HTMLEscapeString em cada interpolacao. Remover a funcao safeHTML (server.go:97-99) para eliminar o footgun.

M-01 - XML injection: campos de formulario interpolados sem escape nos geradores de eventos

MEDIA

Local: internal/web/generator.go:118, 120, 129, 135, 140, 166, 176, 255, 257, 307, 334, 398, 401, 414 | **Categoria:** Inputs sem Tratamento (XSS)

```
fmt.Fprintf(&sb, "      <codAgNoc>%s</codAgNoc>\n", p.CodigoRisco)    // generator.go:118 (sem escapeXML)
fmt.Fprintf(&sb, "      <ideOC>%s</ideOC>\n", orgao)        // generator.go:166
fmt.Fprintf(&sb, "      <dtAcid>%s</dtAcid>\n", p.DtAcidente)    // generator.go:255
```

Por que e exploravel: Parte dos valores de formulario entra no XML por Fprintf sem passar por escapeXML: codAgNoc, tpAval, utilizEPC/eficEpc/utilizEPI, ideOC (orgao de classe), ufOC, dtAcid/hrAcid/dtAtendimento/dtAso/dtExm, ufCRM e ufOC do medico. Um valor como '</codAgNoc><codAgNoc>X' injeta elementos arbitrarios no documento que depois recebe o envelope de assinatura e e validado/transmitido, permitindo adulterar conteudo assinado (por exemplo trocar dados do empregador ou do trabalhador dentro do envelope).

Impacto: Adulteracao de documento fiscal antes da assinatura; assinatura de conteudo divergente do formulario exibido.

Condicao de explorabilidade: Exploravel por POST direto/CSRF; a validacao XSD real (se ligada) reduziria parte dos casos malformados.

Correcao sugerida: Aplicar escapeXML (ou construir o XML com encoding/xml a partir de structs) em TODOS os campos, e validar listas de dominio (datas ISO, tpAval in {1,2}, utilizEPC/EPI in {0,1,2}, UF em lista oficial) antes de gerar.

M-02 - Uploads e corpos de requisicao sem limite de tamanho (consumo de memoria/disco)

MEDIA

Local: internal/web/server.go:409-441, 1051-1067, 701-705 | **Categoria:** Inputs sem Tratamento (XSS)

```
_, _ = io.Copy(destFile, arquivo) // server.go:418 (sem limite)
xmlBytes, err := io.ReadAll(arquivo) // server.go:1067 (sem limite)
linhas, err := leitor.ReadAll() // server.go:705 (sem limite)
// grep "MaxBytesReader|LimitReader" => nenhum resultado no projeto
```

Por que e exploravel: Nao existe http.MaxBytesReader, LimitReader nem ParseMultipartForm com maxMemory em nenhum ponto. O upload de .pfx grava o arquivo inteiro em disco, o import de XML do ASO carrega tudo em memoria e o import de CSV le todas as linhas em memoria. Um cliente (qualquer host da rede, dado o achado C-01) pode esgotar disco ou memoria do processo com um unico request grande.

Impacto: Negacao de servico local (processo morto por OOM / disco cheio) e indisponibilidade do emissor.

Condicao de explorabilidade: Exploravel por qualquer cliente HTTP que alcance a porta.

Correcao sugerida: Envolver o corpo com http.MaxBytesReader (ex.: 2 MB para PFX, 5 MB para XML, 10 MB para CSV), usar r.ParseMultipartForm(limite) e validar Content-Length antes de io.Copy/ReadAll.

B-02 - ID do evento extraido do XML do usuario sem validacao de formato

BAIXA

Local: internal/web/server.go:1277-1284 | **Categoria:** Inputs sem Tratamento (XSS)

```
if strings.Contains(xmlConteudo, "Id=\"") {
    i := strings.Index(xmlConteudo, "Id=\"") + 4
    f := strings.Index(xmlConteudo[i:], "\"")
    if f > 0 { idEvento = xmlConteudo[i : i+f] }
}
```

Por que e exploravel: O identificador do evento passa a ser texto arbitrario do XML enviado pelo usuario e e usado como chave primaria, em URLs (/fila/{id}/xml) e no nome do arquivo de download (Content-Disposition). O html/template escapa o contexto HTML/URL e o net/http substitui CR/LF por espaco em cabecalhos (header.go:139,206), o que impede XSS e header injection; ainda assim nao ha validacao do padrao oficial ID1+tplnsc+nrlnsc+timestamp+seq, o que quebra a integridade referencial do eSocial e permite IDs confusos/duplicados na interface.

Impacto: Dados inconsistentes no eSocial e possibilidade de confusao na operacao (baixo impacto de seguranca direto).

Condicao de explorabilidade: Exploravel por POST /eventos/salvar-generico.

Correcao sugerida: Validar o ID com regex `^ID[0-9]{32}$` (ou regenerar via GerarIDEvento) antes de persistir.

B-05 - Funcao de template safeHTML (template.HTML) definida e nao utilizada

INFORMATIVA

Local: internal/web/server.go:97-99 | **Categoria:** Inputs sem Tratamento (XSS)

```
"safeHTML": func(s string) template.HTML {
    return template.HTML(s)
},
```

Por que e exploravel: A funcao registra um bypass do escaping automatico do html/template. Hoje ela nao e usada em nenhum template (grep por safeHTML em internal/web/templates nao retorna resultado), portanto nao ha vulnerabilidade ativa - mas a existencia dela torna trivial reintroduzir XSS em manutencoes futuras.

Impacto: Risco latente (footgun) de XSS por uso accidental.

Condicao de explorabilidade: Nao exploravel hoje (codigo morto).

Correcao sugerida: Remover a funcao safeHTML ou restringi-la a conteudo gerado internamente, com comentario de alerta.

Complementares (Validacao)**M-03 - Validacao XSD real existe no projeto mas nao esta ligada ao handler; UI afirma validacao oficial**

MEDIA

Local: internal/web/server.go:1433-1436 + internal/web/generator.go:424-482 + internal/esocial/validator.go:101 | **Categoria:** Complementares (Validacao)

```
valido, erros := ValidarEventoXSD(evento.Tipo, evento.XMLGerado) // server.go:1433 (apenas strings.Contains)
// esocial.ValidarXSD (xmllint + XSD oficial) e usado SOMENTE em esocial_test.go
```

Por que e exploravel: esocial.ValidarXSD (validator.go:101) executa xmllint contra os XSD oficiais embutidos, mas nunca e chamada pela camada web. O handler POST /fila/{id}/validar usa ValidarEventoXSD (generator.go:424), que apenas faz strings.Contains de namespace e tags, e devolve a mensagem 'validados com sucesso contra os schemas XSD oficiais' (server.go:1435). O usuario recebe garantia de conformidade que nao foi executada.

Impacto: Falsa sensacao de conformidade; eventos estruturalmente invalidos marcados como validados (retrabalho e rejeicao no eSocial).

Condicao de explorabilidade: Sempre ativo; a mensagem de sucesso e emitida mesmo sem xmllint instalado.

Correcao sugerida: Chamar esocial.ValidarXSD no handler (com fallback documentado quando xmllint nao existe) e ajustar a mensagem ao que realmente foi verificado; opcionalmente embutir um validador Go puro para nao depender do xmllint.

Complementares (Credenciais)

M-04 - Forca bruta da senha do certificado A1 sem autenticao e sem rate limit

MEDIA

Local: internal/web/server.go:214-216, 460-493 | **Categoria:** Complementares (Credenciais)

```
mux.HandleFunc("POST /configuracao/testar", s.handleTestarCertificado) // server.go:216
cert, err := crypto.CarregarA1Arquivo(cfg.CertificadoPath, senha) // server.go:487
```

Por que e exploravel: POST /configuracao/testar aceita a senha do .pfx e responde sucesso/erro de decodificacao, sem autenticao, sem limite de tentativas e sem atraso. Com acesso de rede (achado C-01) e possivel testar senhas indefinidamente contra a chave privada ICP-Brasil armazenada no disco, e a resposta tambem revela titular, CNPJ e emissor do certificado.

Impacto: Comprometimento da chave privada da empresa (e-CNPJ) por dicionario; uso indevido do certificado.

Condicao de explorabilidade: Exploravel por qualquer cliente com acesso a porta 8000.

Correcao sugerida: Exigir autenticao na rota, limitar tentativas por sessao/IP (ex.: 5 tentativas/minuto com backoff), registrar tentativas em log de auditoria e nao devolver detalhes do certificado antes da validacao da senha.

Segredos e Credenciais

B-01 - Permissoes frouxas: PFX gravado com 0644 e diretorio de dados 777 na imagem Docker

BAIXA

Local: internal/web/server.go:413-416 + Dockerfile:16 | **Categoria:** Segredos e Credenciais

```
destFile, err := os.Create(caminhoDest) // server.go:416 -> 0666 & umask = 0644
RUN mkdir -p /app/dados && chmod 777 /app/dados // Dockerfile:16
```

Por que e exploravel: O arquivo .pfx (chave privada) e criado com as permissoes padrao do os.Create (0644 na umask habitual). O diretorio ./dados/certificados e criado com 0700, o que mitiga leitura por outros usuarios locais, mas o Dockerfile torna /app/dados (banco SQLite com CPFs e o proprio PFX) gravavel por qualquer usuario dentro do container, contrariando o principio do menor privilegio.

Impacto: Leitura/adulteracao do banco e do certificado por outro usuario local ou por qualquer processo no container.

Condicao de explorabilidade: Requer outro usuario/processo com acesso ao mesmo volume ou ao container.

Correcao sugerida: Gravar o PFX com os.OpenFile(..., 0600) apos criar o diretorio 0700 (e corrigir permissoes se ja existir); trocar chmod 777 por 0700 no Dockerfile e rodar o container com usuario nao-root (USER 65534).

I-01 - Senhas de teste embutidas no codigo (nenhum segredo de producao encontrado)

INFORMATIVA

Local: internal/crypto/cert_test.go:66, 125 | **Categoria:** Segredos e Credenciais

```
senha := "senhaSegura123" // cert_test.go:66
senha := "testeArquivo456" // cert_test.go:125
```

Por que e exploravel: Sao credenciais de fixtures de teste para PFX gerados em memoria, sem valor em producao. O restante da varredura por segredos (codigo, Dockerfile, workflow, .mcp.json, README e historico git completo) nao encontrou chaves de API, tokens, senhas, chaves privadas ou defaults do tipo \${VAR:-segredo}.

Impacto: Nenhum impacto pratico; registrado apenas como padrao a evitar.

Condicao de explorabilidade: N/A - informativo.

Correcao sugerida: Manter, de preferencia gerando as senhas de fixture em tempo de teste (crypto/rand) para evitar ruido em scanners de segredo.

Complementares (Hardening)

B-03 - Ausencia de cabecalhos de seguranca (CSP, X-Content-Type-Options, X-Frame-Options, Referrer-Policy)

BAIXA

Local: internal/web/server.go:198-257 (Rotas) - nenhum middleware de cabecalhos | **Categoria:** Complementares (Hardening)

```
mux := http.NewServeMux()  
// nenhum w.Header().Set("Content-Security-Policy"|"X-Frame-Options"|"X-Content-Type-Options") no projeto
```

Por que e exploravel: Nenhuma resposta define cabecalhos de seguranca. Sem CSP, o impacto do achado A-03 e amplificado; sem X-Frame-Options/frame-ancestors o painel pode ser embutido em iframe (clickjacking) e sem X-Content-Type-Options o navegador pode farejar tipos de conteudo.

Impacto: Agravamento de XSS/clickjacking; defesa em profundidade ausente.

Condicao de explorabilidade: Sempre presente (ausencia de controle).

Correcao sugerida: Middleware unico aplicando CSP restritiva (default-src 'self'), X-Content-Type-Options: nosniff, X-Frame-Options: DENY e Referrer-Policy: no-referrer.

Complementares (Conformidade)

M-05 - XML gerado pelo editor nao e aderente ao XSD oficial S-1.3 (revelado ao ligar a validacao real)

ALTA

Local: internal/web/generator.go:97-181, 239-342, 381-421 + internal/data/xsd/*.xsd | **Categoria:** Complementares (Conformidade)

```
gerador emite <ideTrabalhador><cpfTrab> // evtExpRisco.xsd:31 exige <ideVinculo type="T_ideVinculo_sst">  
gerador emite <agenteNoc> // evtExpRisco.xsd exige <agNoc>  
gerador emite <ideOC>CREA</ideOC> // XSD: enumeracao {1,4,9} (codigo do orgao, nao sigla)  
// xmllint --schema evtExpRisco.xsd: "element ideTrabalhador: This element is not expected. Expected is ( ideVinculo )"
```

Por que e exploravel: Ate esta correcao o botao Validar apenas fazia strings.Contains e sempre aprovava, mascarando a divergencia. Ao ligar o validador real (esocial.ValidarXSD com xmllint + XSD oficiais embutidos), ficou demonstrado que os documentos produzidos pelo editor nao sao aderentes ao leiaute S-1.3: nomes de grupos diferentes (ideTrabalhador vs ideVinculo, agenteNoc vs agNoc), epi sem o subgrupo epiCompl e ideOC textual em vez de codigo. O mesmo vale para S-2210 (faltam codSitGeradora, iniciatCAT, lateralidade, indInternacao, durTrat, indAfast) e S-2220 (exMedOcup/aso/resAso).

Impacto: Conformidade: os eventos nao seriam aceitos pelo eSocial real. Com a correcao do achado M-03 o sistema passa a reprovar de forma explicita e rastreavel, em vez de aprovar falsamente. Severidade: ALTA (funcional/regulatorio).

Condicao de explorabilidade: Visivel em qualquer evento gerado pelo editor e submetido a conferencia por schema.

Correcao sugerida: Adequar os tres geradores ao XSD oficial embarcado (usar ideVinculo com matricula, agNoc, bloco epi/epiCompl, ideOC como codigo, e completar os grupos obrigatorios de CAT e ASO), com teste de regressao que valide cada evento gerado com xmllint contra o schema embutido.

5. Recomendacoes priorizadas

Prio.	Acao	Detalhe	Achados
P1	Fechar a fronteira de autorizacao	Trocar o bind default para 127.0.0.1, adicionar autenticao com middleware global, validar o cabecalho Host e aplicar token anti-CSRF em todos os POSTs. Corrige C-01 e A-01.	C-01, A-01
P1	Parar de falsificar assinatura e recibos	Marcar explicitamente o modo simulacao, nao gravar recibo/protocolo sem transmissao real, conectar o ClienteSOAP mTLS e corrigir o README. Corrige A-02.	A-02
P2	Eliminar a saida HTML manual	Migrar o handler de certificado para template.ExecuteTemplate com html/template, remover safeHTML e cobrir com teste de regressao o nome de arquivo e os campos do certificado. Corrige A-03 e B-05.	A-03, B-05
P2	Blindar a geracao de XML	Aplicar escapeXML/encoding/xml em todos os campos e validar dominios (datas ISO, tpAval, utilizEPC/EPI, UF). Corrige M-01.	M-01
P2	Limitar entrada e abuso	http.MaxBytesReader em uploads e corpos, limites por tipo de arquivo e rate limit no teste de senha do certificado. Corrige M-02 e M-04.	M-02, M-04
P3	Ligar a validacao real	Chamar esocial.ValidarXSD no handler e ajustar a mensagem ao que foi verificado. Corrige M-03.	M-03
P3	Hardening de arquivos e imagem	PFX com 0600, diretorio de dados 0700, container sem root, senhas de teste geradas em runtime. Corrige B-01 e I-01.	B-01, I-01
P3	Defesa em profundidade e observabilidade	Cabecalhos de seguranca, validacao do ID do evento e tratamento de erros de persistencia com feedback real. Corrige B-02, B-03 e B-04.	B-02, B-03, B-04

7. Status das correcoes aplicadas

Esta secao registra o que foi efetivamente alterado no codigo para cada achado, com a evidencia de verificacao. As correcoes foram validadas por **go build**, **go vet**, suite de testes (**go test ./...**, incluindo os novos testes de regressao em **internal/web/seguranca_test.go**) e teste ponta a ponta com o binario real (login, sessao, CSRF, Host, cabecalhos, limites de upload e fluxo de simulacao).

Achado	Status	Correcao aplicada	Evidencia
C-01	CORRIGIDO	Bind padrao em 127.0.0.1 (flag -host), autenticacao local obrigatoria (senha com hash SHA-256 iterado e salt, cookie HttpOnly + SameSite=Strict, sessao de 12h), middleware de auth em todas as rotas exceto /login e /static, allowlist de Host (anti DNS rebinding) e tela de login com logout.	internal/web/auth.go (novo), internal/web/templates/login.html (novo), internal/web/server.go (Rotas + middleware), cmd/server/main.go (-host, -senha, ESOCIAL_SENHA); testes TestRotasExigemAutenticacao e TestHostNaoAutorizadoRejeitado
A-01	CORRIGIDO	Token anti-CSRF por processo embutido em todos os formularios e no cabecalho X-CSRF-Token das requisicoes HTMX (hx-headers), validado em todo metodo de escrita junto com Origin/Referer.	internal/web/auth.go (middlewareCSRF), 13 formularios em internal/web/templates/*.html; testes TestCSRFExigidoEmEscrita
A-02	CORRIGIDO	Assinatura simulada marcada explicitamente (comentario + base64 legivel SIMULACAO-...) e sem alegacao de validade; transmissao passa a gravar status 'simulado' sem protocolo e sem recibo; consulta de recibo nao inventa numero; aviso permanente MODO SIMULACAO na interface; README corrigido.	internal/web/generator.go (SimularAssinatura/MensagemTransmissaoSimulada), internal/web/server.go (assinar/transmitir/recibo/lotos), internal/web/templates/layout.html, README.md; testes TestAssinaturaSimuladaMarcadaENenhumReciboFabricado e TestEventosSSTCicloDeVidaFila
A-03	CORRIGIDO	Saida do teste de certificado migrada de fmt.Fprintf para html/template (escaping contextual), com partial dedicado; remocao da funcao safeHTML.	internal/web/server.go (renderTesteCertificado + DadosTesteCertificado), internal/web/templates/partials/teste_certificado.html (novo); teste TestXSSNomeArquivoCertificadoEscapado
M-01	CORRIGIDO	Todos os campos interpolados no XML passam por escapeXML ou validacao de dominio (datas ISO, hora HHMM, tpAval, utilizEPC/EPI, UF oficial, tpAcid, tpExameOcup).	internal/web/generator.go (validarDataISO, validarHora, validarDominio, validarUF); teste TestGeradorXMLEscapaCamposMaliciosos
M-02	CORRIGIDO	http.MaxBytesReader + limite por tipo de arquivo (2 MB PFX, 5 MB XML, 10 MB CSV), leitura com io.LimitReader e mensagem explicita ao usuario quando o limite e excedido.	internal/web/server.go (constantes limiteUpload* e handlers de upload), internal/web/auth.go (limite global de 12 MB para escritas); teste TestUploadAcimaDoLimite
M-03	CORRIGIDO	Validacao por schema oficial (esocial.ValidarXSD + xmllint + XSD embutidos) ligada ao handler, com fallback declarado apenas para tipos sem schema e mensagem que informa o modo executado.	internal/web/server.go (handleValidarEvento), internal/esocial/validator.go (XmllintDisponivel); teste TestValidacaoUsaSchemaOficial
M-04	CORRIGIDO	Rate limit por IP no login (8/min) e na validacao da senha do certificado (6/min), com resposta 429/Retry-After e registro em log de auditoria.	internal/web/auth.go (limitador, permitirTesteCertificado); teste TestRotasExigemAutenticacao (rota protegida)
B-01	CORRIGIDO	PFX gravado com 0600 e diretorio de certificados corrigido para 0700; banco SQLite e seu diretorio com 0700/0600; imagem Docker com chmod 700 e execucao como usuario nao-root (UID 10001).	internal/web/server.go (upload), internal/storage/storage.go (Abrir), Dockerfile (USER 10001 + chmod 700); testes TestPermissaoDoArquivoDeCertificado e verificacao em execucao real (ls -l dados/)
B-02	CORRIGIDO	ID do evento validado contra o padrao oficial ID+tpInsc+nrInsc(14)+timestamp(14)+seq(5); ID invalido e substituido por um ID gerado, com aviso ao operador.	internal/web/server.go (idEventoValido, extrairIDEvento); teste TestIDEventoInvalidoSubstituido

Achado	Status	Correcao aplicada	Evidencia
B-03	CORRIGIDO	Middleware de cabecalhos de seguranca: CSP (default-src 'self', frame-ancestors 'none', object-src 'none', base-uri 'none', form-action 'self'), X-Content-Type-Options, X-Frame-Options, Referrer-Policy e COOP.	internal/web/auth.go (middlewareCabecalhos); teste TestCabecalhosDeSeguranca + verificacao com curl no binario real
B-04	CORRIGIDO	Erros de persistencia deixaram de ser descartados: exclusoes verificam existencia e falha de gravacao/exclusao gera flash de erro em vez de mensagem de sucesso.	internal/web/server.go (handleExcluirEvento, handleExcluirColaborador, handleSalvarS2210/S2220, handleImportarXMLASO, handleSalvarEventoGenerico); teste TestExclusaoInexistenteInformaErro
B-05	CORRIGIDO	Funcao safeHTML (template.HTML) removida do FuncMap.	internal/web/server.go (carregarTemplates); teste TestSemFuncaoSafeHTML
I-01	CORRIGIDO	Senhas de fixture dos testes geradas em tempo de execucao com crypto/rand (nenhum literal de credencial no codigo).	internal/crypto/cert_test.go (senhaAleatoriaTeste); go test ./internal/crypto

Achado adicional (aberto): **M-05** - os geradores de XML do editor ainda nao sao aderentes ao XSD oficial S-1.3 (ideVinculo, agNoc, codigos de orgao de classe e blocos obrigatorios de CAT/ASO). A correcao do M-03 tornou essa divergencia visivel e rastreavel em vez de silenciosa; a adequacao dos leiautes depende de decisao funcional sobre os campos obrigatorios do documento fiscal (issue dedicada na secao 6).

6. ISSUES PARA O GITHUB

Os blocos abaixo estao prontos para copiar e colar na criacao de issues. Cada bloco vai do delimitador **--- ISSUE n ---** ate **--- FIM ISSUE n ---** e contem titulo, labels sugeridas, descricao, evidencia, impacto, correcao e criterios de aceite em Markdown. Achados triviais relacionados foram agrupados para evitar spam de issues.

Issue 1 de 11

--- ISSUE 1 ---

[Seguranca] Adicionar autenticao/autorizacao e restringir o bind de rede (0.0.0.0:8000)

Labels: security, critical, area: backend

Descricao

O servidor web nao possui nenhuma camada de autenticao ou autorizacao e escuta em todas as interfaces de rede.

- `cmd/server/main.go:55` -> `Addr: fmt.Sprintf(":%d", \*porta)` (bind 0.0.0.0, nao 127.0.0.1)
- `internal/web/server.go:198-257` -> `Rotas()` registra 30+ handlers sem qualquer middleware de sessao/login/token

Sonda contra as rotas reais (sem nenhuma credencial):

```
...
GET /fila -> 200
GET /colaboradores -> 200
GET /configuracao -> 200
DELETE /colaboradores/{id} -> 200
POST /fila/lote/transmitir -> 200
...
```

Por que e exploravel

Qualquer host que alcance a porta 8000 (LAN, Wi-Fi publico, container com `-p 8000:8000`) pode ler a lista de colaboradores com CPF, baixar os XMLs dos eventos, alterar razao social/CNPJ/ambiente, substituir o certificado digital, disparar assinatura/transmissao em lote e apagar registros. Nao ha nenhum controle de acesso a contornar.

Impacto

Exposicao de dados pessoais (LGPD) de toda a folha, manipulacao de documentos fiscais e destruicao de dados.
Severidade: CRITICA.

Sugestao de correcao

1. Bind padrao em `127.0.0.1`, com flag explicita (`-expor`) para outros enderecos.
2. Autenticacao local (sessao com cookie `SameSite=Strict` + `HttpOnly`, ou token de dispositivo) e middleware aplicado a todas as rotas exceto `/static/`.
3. Validacao do cabeçalho `Host` com allowlist (`localhost`, `127.0.0.1`) para mitigar DNS rebinding.
4. Docker: documentar o risco e exigir auth quando a porta for publicada.

Criterios de aceite

```
- [ ] `curl -i http://127.0.0.1:8000/fila` sem credencial retorna 401/302 para login
- [ ] Nenhuma rota de escrita responde 200 sem sessao valida
- [ ] `Addr` default e `127.0.0.1:8000` (teste automatizado cobrindo o default)
- [ ] Requisicao com `Host: evil.test` e rejeitada
- [ ] README documenta o modo de exposicao e a necessidade de auth
--- FIM ISSUE 1 ---
```


Issue 2 de 11

--- ISSUE 2 ---

[Seguranca] Proteger endpoints de escrita contra CSRF (token + validacao de Origin)

Labels: security, high, area: backend

Descricao

Nenhum formulario possui token anti-CSRF e nenhum handler valida `Origin`/`Referer`.

```
- `internal/web/server.go:213-251` -> rotas `POST /configuracao`, `POST /colaboradores`, `POST /eventos/*`,  
  `POST /fila/{id}/assinar`, `POST /fila/{id}/transmitir`, `POST /fila/lote/assinar`, `POST /fila/lote/transmitir`  
- `grep -rni "csrf" .` -> nenhum resultado no repositorio
```

Por que e exploravel

POST de formulario e requisicao simples (sem preflight CORS). Uma pagina maliciosa visitada pelo operador pode enviar formularios para `http://localhost:8000` e executar acoes de escrita: trocar ambiente para Producao Oficial, alterar CNPJ, cadastrar colaborador, assinar/transmitir lotes inteiros.

Impacto

Integridade da configuracao e dos dados fiscais; acoes privilegiadas disparadas pelo navegador da vitima. Severidade: ALTA.

Sugestao de correcao

- Token sincronizado por formulario (campo `hidden`) validado no servidor para todo metodo nao-idempotente; ou
- Exigir cabecalho customizado (`X-CSRF-Token`) e validar `Origin`/`Referer` contra o host da aplicacao.
- Com sessao, usar cookie `SameSite=Strict`.

Criterios de aceite

- [] POST sem token/cabecalho valido retorna 403
- [] Todos os formularios em `internal/web/templates/` incluem o campo de token
- [] POST com `Origin: https://evil.test` e rejeitado (teste automatizado)
- [] Teste de regressao cobrindo `POST /configuracao` e `POST /fila/lote/transmitir`

--- FIM ISSUE 2 ---

Issue 3 de 11

--- ISSUE 3 ---

[Seguranca] Remover assinatura e recibos simulados ou rotula-los explicitamente

Labels: security, high, integrity, area: core

Descricao

A aplicacao grava assinatura XMLDSig falsa, inventa protocolo/recibo e marca o evento como `aceito`, apresentando isso como retorno oficial do governo.

```
- `internal/web/generator.go:503` -> `<SignatureValue>SIMULATED_RSA_SHA256_SIGNATURE_...`  
- `internal/web/generator.go:516-533` -> `SimilarTransmissao` gera protocolo/recibo numericos e a mensagem  
  "Recibo emitido pelo Serpro/Receita Federal"  
- `internal/web/server.go:1491` -> "Recibo oficial do eSocial ja confirmado: %s"  
- `internal/soap/client.go` (mTLS real) nao e referenciado por nenhuma rota - codigo morto  
- `README.md:14,65-67` afirma assinatura digital e transmissao pelos webservices oficiais
```

Por que e exploravel

Nao e um bypass de permissao: e uma funcionalidade que produz prova documental inexistente. O operador acredita ter cumprido obrigacao acessoria do eSocial quando nada foi assinado nem transmitido.

Impacto

Trilha de auditoria fiscal falsa, risco regulatorio/legal e assinatura sem validade juridica. Severidade: ALTA.

Sugestao de correcao

1. Introduzir status explicito de simulacao (`simulado`) e nao preencher `recibo`/`protocolo` nesse modo.
2. Rotular na interface ("modo demonstracao - sem transmissao real").
3. Conectar `soap.ClienteSOAP` (mTLS + A1) aos handlers, persistindo o XML de resposta oficial.
4. Corrigir o README para refletir o estado real da funcionalidade.

Criterios de aceite

- [] Evento sem transmissao real nunca recebe status `aceito` nem campo `recibo` preenchido
- [] Interface exhibe aviso claro no modo simulacao
- [] Fluxo real de envio/consulta usa `internal/soap` com certificado A1 (teste de integracao com mock)
- [] README sem afirmacoes de transmissao nao implementada

--- FIM ISSUE 3 ---

Issue 4 de 11

--- ISSUE 4 ---

[Seguranca] Corrigir HTML injection/XSS armazenado em POST /configuracao/testar

Labels: security, high, xss, area: frontend

Descricao

O handler responde HTML montado por `fmt.Fprintf`, interpolando dados controlados pelo usuario sem escape.

- `internal/web/server.go:472-474` -> `filepath.Base(cfg.CertificadoPath)` (nome do .pfx enviado no upload, persistido no SQLite)
- `internal/web/server.go:489-492` -> `err.Error()` de decodificacao do PKCS#12
- `internal/web/server.go:518-539` -> titular, CNPJ, emissor e datas vindos do certificado carregado

Evidencia de sonda (httptest contra a rota real, `CertificadoPath = "<img src=x onerror=alert(1)>.pfx"`):

...

STATUS=200

BODY=... O arquivo .pfx nao foi localizado no disco local.

PROBE_RESULT=HTML_INJECTION_CONFIRMED

...

Por que e exploravel

O valor persiste no banco e e refletido em cada visita a pagina de certificado: e XSS armazenado na origem do painel (localhost:8000). Combinado com o achado de CSRF, o payload pode ser plantado por uma pagina externa.

Impacto

Execucao de JavaScript na origem do painel, com leitura de todo o conteudo e disparo de acoes autenticadas pelo navegador do operador. Severidade: ALTA.

Sugestao de correcao

- Substituir todo `fmt.Fprintf` de HTML por `template.ExecuteTemplate` (escaping contextual) ou aplicar `template.HTMLEscapeString` em cada interpolacao.
- Remover a funcao `safeHTML` (`internal/web/server.go:97-99`), hoje morta, para eliminar o bypass latente.

Criterios de aceite

- [] Teste automatizado: nome de arquivo `<img src=x onerror=alert(1)>.pfx` nao produz tag HTML na resposta
- [] Nenhum `fmt.Fprintf(w, "<...")` restante em `internal/web/`
- [] Funcao `safeHTML` removida
- [] Teste cobrindo tambem CN/titular de certificado malicioso

--- FIM ISSUE 4 ---

Issue 5 de 11

--- ISSUE 5 ---

[Seguranca] Escapar todos os campos nos geradores de XML (XML injection)

Labels: security, medium, area: core

Descricao

Campos de formulario sao interpolados no XML sem `escapeXML`, permitindo injetar elementos arbitrarios no documento que depois recebe assinatura/transmissao.

```
- `internal/web/generator.go:118` -> `<codAgNoc>%s</codAgNoc>` com `p.CodigoRisco`  
- `internal/web/generator.go:120` -> `<tpAval>%s</tpAval>`  
- `internal/web/generator.go:129,135,140` -> `utilizEPC`, `eficEpc`, `utilizEPI`  
- `internal/web/generator.go:166,176` -> `ideOC`, `ufOC`  
- `internal/web/generator.go:255,257,307,398,401` -> datas e horarios  
- `internal/web/generator.go:334,414` -> `ufOC`/`ufCRM` do medico
```

Por que e exploravel

Um valor como `</codAgNoc><codAgNoc>X` injeta estrutura nova no XML. Se o conteudo injetado for valido para o schema, ele passa a fazer parte do documento assinado.

Impacto

Adulteracao de documento fiscal antes da assinatura; assinatura de conteudo divergente do formulario exibido. Severidade: MEDIA.

Sugestao de correcao

- Aplicar `escapeXML` em todos os campos (ou construir o XML com `encoding/xml` a partir de structs).
- Validar dominios: datas ISO, `tpAval` em {1,2}, `utilizEPC/EPI` em {0,1,2}, UF em lista oficial.

Criterios de aceite

- [] Teste de regressao com payload `</tag><tag>X` em cada campo de texto nao altera a estrutura do XML
- [] Todos os `Fprintf(&sb, ...)` de valor usam escape ou dominio validado
- [] Validacao XSD real (issue propria) ligada ao fluxo de gravacao

--- FIM ISSUE 5 ---

Issue 6 de 11

--- ISSUE 6 ---

[Seguranca] Aplicar limites de tamanho em uploads e corpos de requisicao

Labels: security, medium, dos, area: backend

Descricao

Nao ha limite de tamanho em uploads nem em leitura de corpo.

```
- `internal/web/server.go:418` -> `io.Copy(destFile, arquivo)` (PFX, sem limite)  
- `internal/web/server.go:1067` -> `io.ReadAll(arquivo)` (XML do ASO, sem limite)  
- `internal/web/server.go:705` -> `leitor.ReadAll()` (CSV, sem limite)  
- `grep -rn "MaxBytesReader\|LimitReader" .` -> nenhum resultado
```

Por que e exploravel

Qualquer cliente que alcance a porta (ver issue de autenticacao) pode enviar um arquivo arbitrariamente grande e esgotar disco ou memoria do processo.

Impacto

Negacao de servico local (OOM/disco cheio). Severidade: MEDIA.

Sugestao de correcao

- `http.MaxBytesReader` nos handlers de upload (2 MB PFX, 5 MB XML, 10 MB CSV).
- `r.ParseMultipartForm(limite)` e validacao de `Content-Length` antes de `io.Copy`/`ReadAll`.
- Retornar 413 com mensagem clara ao usuario.

Criterios de aceite

- [] Upload de arquivo acima do limite retorna 413
- [] Testes automatizados cobrindo PFX, XML e CSV
- [] Nenhum `io.Copy`/`io.ReadAll` de corpo HTTP sem leitor limitado

--- FIM ISSUE 6 ---

Issue 7 de 11

--- ISSUE 7 ---

[Seguranca] Conectar a validacao XSD real (xmllint) ao handler de validacao

Labels: security, medium, correctness, area: backend

Descricao

O validador XSD oficial existe no projeto, mas o handler usa apenas checagem por substring e afirma validacao oficial.

```
- `internal/web/server.go:1433` -> `ValidarEventoXSD(evento.Tipo, evento.XMLGerado)`  
- `internal/web/generator.go:424-482` -> apenas `strings.Contains` de namespace/tags  
- `internal/web/server.go:1435` -> mensagem "validados com sucesso contra os schemas XSD oficiais"  
- `internal/esocial/validator.go:101` -> `ValidarXSD` (xmllint + XSD S-1.3) usado SOMENTE em `esocial_test.go`
```

Por que e exploravel

Nao ha barreira tecnica: a funcionalidade simplesmente nao foi ligada. O usuario recebe garantia de conformidade que nao foi executada e eventos invalidos seguem para assinatura/transmissao.

Impacto

Falsa sensacao de conformidade, retrabalho e rejeicao de eventos no eSocial. Severidade: MEDIA.

Sugestao de correcao

```
- Chamar `esocial.ValidarXSD` no `handleValidarEvento` e ajustar a mensagem ao que foi efetivamente verificado.  
- Documentar o fallback quando `xmllint` nao estiver instalado (ou embutir validador Go puro).
```

Criterios de aceite

```
- [ ] POST /fila/{id}/validar executa a validacao por schema (xmllint) quando disponivel  
- [ ] Mensagem de sucesso so aparece apos validacao real; caso contrario informa modo degradado  
- [ ] Teste automatizado com XML invalido (schema) retornando rejeicao  
--- FIM ISSUE 7 ---
```

Issue 8 de 11

--- ISSUE 8 ---

[Seguranca] Limitar tentativas de senha do certificado A1 (anti forca bruta)

Labels: security, medium, area: backend

Descricao

`POST /configuracao/testar` aceita a senha do .pfx sem autenticacao, sem rate limit e sem atraso, e devolve dados do certificado.

```
- `internal/web/server.go:216` -> rota registrada sem middleware  
- `internal/web/server.go:487` -> `crypto.CarregarA1Arquivo(cfg.CertificadoPath, senha)`
```

Por que e exploravel

Com acesso de rede (ver issue de autenticacao), e possivel testar senhas indefinidamente contra a chave privada ICP-Brasil armazenada no disco; a resposta revela titular, CNPJ e emissor.

Impacto

Comprometimento da chave privada da empresa (e-CNPJ) por dicionario. Severidade: MEDIA.

Sugestao de correcao

```
- Exigir autenticacao na rota e limitar tentativas por sessao/IP (ex.: 5/min com backoff).  
- Registrar tentativas em log de auditoria.  
- Nao expor dados do certificado antes da validacao da senha.
```

Criterios de aceite

```
- [ ] Apos N tentativas invalidas a rota retorna 429 com backoff  
- [ ] Tentativas registradas em log com IP e horario  
- [ ] Rota exige sessao autenticada  
--- FIM ISSUE 8 ---
```

Issue 9 de 11

--- ISSUE 9 ---

[Seguranca/Conformidade] Adequar os geradores de eventos ao XSD oficial S-1.3

Labels: security, high, correctness, area: core

Descricao

Com a validacao por schema agora ativa (issue M-03), ficou demonstrado que os XML gerados pelo editor nao sao aderentes aos XSD oficiais embarcados.

```
- `internal/web/generator.go:99-101` -> `  exige `- `internal/web/generator.go:117-158` -> `- `internal/web/generator.go:166` -> `- bloco `epi` sem o subgrupo obrigatorio `epiCompl`  
- S-2210: faltam `codSitGeradora`, `iniciatCAT`, `lateralidade`, `indInternacao`, `durTrat`, `indAfast`  
- S-2220: estrutura `exMedOcup`/`aso`/`resAso` divergente
```

Evidencia (`xmllint --noout --schema internal/data/xsd/evtExpRisco.xsd evento.xml`):

```
...
```

```
element ideTrabalhador: This element is not expected. Expected is ( ideVinculo ).
```

```
...
```

Por que e exploravel

Ate a correcao do achado M-03 o botao Validar aprovava qualquer documento (apenas `strings.Contains`), o que mascarava a divergencia e levava o operador a assinar um XML que o eSocial rejeitaria.

Impacto

Eventos rejeitados no envio real e falsa sensacao de conformidade. Severidade: ALTA (funcional/regulatorio).

Sugestao de correcao

1. Adequar os tres geradores (`GerarXMLS2240`, `GerarXMLS2210`, `GerarXMLS2220`) ao XSD embarcado, com os grupos e vocabularios obrigatorios.
2. Adicionar teste de regressao que valide o XML de cada gerador com `xmllint --schema` (skip quando o binario nao existir).
3. Manter a validacao por schema no fluxo (issue M-03) para evitar regressao silenciosa.

Criterios de aceite

```
- [ ] `xmllint --schema internal/data/xsd/evtExpRisco.xsd` aprova a saida de `GerarXMLS2240`  
- [ ] `xmllint --schema internal/data/xsd/evtCAT.xsd` aprova a saida de `GerarXMLS2210`  
- [ ] `xmllint --schema internal/data/xsd/evtMonit.xsd` aprova a saida de `GerarXMLS2220`  
- [ ] Teste automatizado cobre os tres geradores (com skip documentado se `xmllint` nao estiver instalado)  
--- FIM ISSUE 9 ---
```

Issue 10 de 11

--- ISSUE 10 ---

[Seguranca] Endurecer permissoes de arquivos, dados de teste e imagem Docker

Labels: security, low, hardening

Descricao

Achados de baixa severidade agrupados no mesmo tema (arquivos/credenciais):

- `internal/web/server.go:416` -> `os.Create` grava o .pfx com 0644 (chave privada legivel por outros usuarios locais)
- `Dockerfile:16` -> `chmod 777 /app/dados` (banco SQLite com CPFs e o proprio PFX gravaveis por qualquer processo)
- `internal/crypto/cert\_test.go:66,125` -> senhas de teste literais ("senhaSegura123", "testeArquivo456")

Observacao: a varredura completa por segredos (codigo, CI, Dockerfile, docs, `.mcp.json` e historico git) nao encontrou nenhum segredo de producao ou default inseguro do tipo `\${VAR:-segredo}`.

Impacto

Leitura/adulteracao do banco e do certificado por outro usuario local ou processo no container; ruído em scanners de segredo. Severidade: BAIXA.

Sugestao de correcao

- Gravar o PFX com `os.OpenFile(..., 0600)` e corrigir permissoes de diretorio existente.
- Trocar `chmod 777` por `0700` e rodar o container com `USER` nao-root.
- Gerar senhas de fixture em tempo de teste (crypto/rand).

Criterios de aceite

- [] Arquivo .pfx criado com 0600 (teste verificando `os.Stat().Mode()`)
 - [] Dockerfile sem `chmod 777` e com usuario nao-root
 - [] Nenhuma senha literal em `internal/crypto/\*\_test.go`
- FIM ISSUE 10 ---

Issue 11 de 11

--- ISSUE 11 ---

[Seguranca] Adicionar cabecalhos de seguranca, validar ID de evento e tratar erros de persistencia

Labels: security, low, hardening, area: backend

Descricao

Achados de baixa severidade agrupados:

1. Sem cabecalhos de seguranca em nenhuma resposta (`internal/web/server.go:198-257`): falta CSP, `X-Content-Type-Options`, `X-Frame-Options` e `Referrer-Policy`.
2. ID de evento vem do XML do usuario sem validacao (`internal/web/server.go:1277-1284`), virando chave primaria, segmento de URL e nome de arquivo de download. O html/template escapa os contextos HTML/URL e o net/http substitui CR/LF em cabecalhos (`net/http/header.go:139,206`), entao nao ha XSS/header injection, mas o padrao oficial `ID1+tpInsc+nrInsc+timestamp+seq` nao e respeitado.
3. Erros de persistencia sao descartados com flash de sucesso (`internal/web/server.go:660, 966, 1045, 1113, 1297, 1502`).

Impacto

Defesa em profundidade ausente, dados inconsistentes e trilha de auditoria enganosa. Severidade: BAIXA.

Sugestao de correcao

- Middleware aplicando CSP `default-src 'self'`, `nosniff`, `DENY` e `no-referrer`.
- Validar o ID com `^ID[0-9]{32}\$` (ou sempre usar `GerarIDEvento`).
- Tratar os erros retornados, retornando 404/500 coerentes e flash de erro.

Criterios de aceite

- [] Todas as respostas HTML incluem CSP e demais cabecalhos (teste automatizado)
 - [] ID fora do padrao e rejeitado com 400
 - [] Nenhum `s.db.` restante em `internal/web/server.go`
 - [] Falha simulada de persistencia exibe mensagem de erro ao usuario
- FIM ISSUE 11 ---